

```
!pip install tensorflow
```

```
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Collecting google_pasta>=0.1.1 (from tensorflow)
  Downloading google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Collecting libclang>=13.0.0 (from tensorflow)
  Downloading libclang-18.1.1-py2.py3-none-manylinux2010_x86_64.whl.metadata (5.2 kB)
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Collecting tensorboard~2.20.0 (from tensorflow)
  Downloading tensorboard-2.20.0-py3-none-any.whl.metadata (1.8 kB)
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Collecting wheel<1.0,>=0.23.0 (from astunparse>=1.6.0->tensorflow)
  Downloading wheel-0.46.3-py3-none-any.whl.metadata (2.4 kB)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from rich)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~2.20.0)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from tensorboard~2.20.0)
Collecting tensorboard_data_server<0.8.0,>=0.7.0 (from tensorboard~2.20.0->tensorflow)
  Downloading tensorboard_data_server-0.7.2-py3-none-manylinux_2_31_x86_64.whl.metadata (1.1 kB)
Collecting werkzeug>=1.0.1 (from tensorboard~2.20.0->tensorflow)
  Downloading werkzeug-3.1.5-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from werkzeug)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py)
Requirement already satisfied: mdurl~0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py)
Downloading tensorflow-2.20.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (620.7 MB)
  620.7/620.7 MB 757.4 kB/s eta 0:00:00
Downloading astunparse-1.6.3-py2.py3-none-any.whl (12 kB)
Downloading flatbuffers-25.12.19-py2.py3-none-any.whl (26 kB)
Downloading google_pasta-0.2.0-py3-none-any.whl (57 kB)
  57.5/57.5 kB 6.3 MB/s eta 0:00:00
Downloading libclang-18.1.1-py2.py3-none-manylinux2010_x86_64.whl (24.5 MB)
  24.5/24.5 MB 103.8 MB/s eta 0:00:00
Downloading tensorboard-2.20.0-py3-none-any.whl (5.5 MB)
  5.5/5.5 MB 155.4 MB/s eta 0:00:00
Downloading tensorboard_data_server-0.7.2-py3-none-manylinux_2_31_x86_64.whl (6.6 MB)
  6.6/6.6 MB 163.2 MB/s eta 0:00:00
Downloading werkzeug-3.1.5-py3-none-any.whl (225 kB)
  225.0/225.0 kB 24.0 MB/s eta 0:00:00
Downloading wheel-0.46.3-py3-none-any.whl (30 kB)
Installing collected packages: libclang, flatbuffers, wheel, werkzeug, tensorboard-data-server, google_pasta, tensorflow
Successfully installed astunparse-1.6.3 flatbuffers-25.12.19 google_pasta-0.2.0 libclang-18.1.1 tensorflow-2.20.0
```

```
import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt

# Load MNIST
(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()

# Normalize to [-1,1]
x_train = (x_train - 127.5) / 127.5
x_train = np.expand_dims(x_train, axis=-1)

BUFFER_SIZE = 60000
```

```
BATCH_SIZE = 128
```

```
dataset = tf.data.Dataset.from_tensor_slices(x_train)
dataset = dataset.shuffle(BUFFER_SIZE).batch(BATCH_SIZE)
```

```
print("Dataset Ready:", x_train.shape)
```

```
/usr/local/lib/python3.12/dist-packages/jax/_src/cloud_tpu_init.py:86: UserWarning: Transparent hugepage
warnings.warn(
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step
```

```
Dataset Ready: (60000, 28, 28, 1)
```

```
def build_generator():
```

```
    model = tf.keras.Sequential()
```

```
    model.add(layers.Dense(7*7*256, use_bias=False, input_shape=(100,)))
```

```
    model.add(layers.BatchNormalization())
```

```
    model.add(layers.LeakyReLU())
```

```
    model.add(layers.Reshape((7, 7, 256)))
```

```
    model.add(layers.Conv2DTranspose(128, (5,5), strides=(1,1), padding='same', use_bias=False))
```

```
    model.add(layers.BatchNormalization())
```

```
    model.add(layers.LeakyReLU())
```

```
    model.add(layers.Conv2DTranspose(64, (5,5), strides=(2,2), padding='same', use_bias=False))
```

```
    model.add(layers.BatchNormalization())
```

```
    model.add(layers.LeakyReLU())
```

```
    model.add(layers.Conv2DTranspose(1, (5,5), strides=(2,2), padding='same', use_bias=False, activation='sigmoid'))
```

```
    return model
```

```
generator = build_generator()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
def build_discriminator():
```

```
    model = tf.keras.Sequential()
```

```
    model.add(layers.Conv2D(64, (5,5), strides=(2,2), padding='same',
                           input_shape=[28,28,1]))
```

```
    model.add(layers.LeakyReLU())
```

```
    model.add(layers.Dropout(0.3))
```

```
    model.add(layers.Conv2D(128, (5,5), strides=(2,2), padding='same'))
```

```
    model.add(layers.LeakyReLU())
```

```
    model.add(layers.Dropout(0.3))
```

```
    model.add(layers.Flatten())
```

```
    model.add(layers.Dense(1, activation='sigmoid'))
```

```
    return model
```

```
discriminator = build_discriminator()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
cross_entropy = tf.keras.losses.BinaryCrossentropy()
```

```
def discriminator_loss(real_output, fake_output):
```

```
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
```

```

fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

```

```

generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)

```

```

@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, 100])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, discriminator.trainable_variables))

```

```

import matplotlib.pyplot as plt

def generate_and_plot(model, epoch, test_input):
    predictions = model(test_input, training=False)

    plt.figure(figsize=(4,4))
    for i in range(predictions.shape[0]):
        plt.subplot(4,4,i+1)
        plt.imshow(predictions[i, :, :, 0] * 127.5 + 127.5, cmap='gray')
        plt.axis('off')

    plt.suptitle(f"Generated Images - Epoch {epoch}")
    plt.show()

```

```

EPOCHS = 3
noise_seed = tf.random.normal([16, 100])

# Modify train_step to return losses
@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, 100])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, discriminator.trainable_variables))

```

```
        return gen_loss, disc_loss

for epoch in range(EPOCHS):

    print("\n=====")
    print(f"Starting Epoch {epoch+1}/{EPOCHS}")
    print("=====")

    for step, image_batch in enumerate(dataset):
        gen_loss, disc_loss = train_step(image_batch)

        if step % 200 == 0:
            print(f"Batch {step} | Gen Loss: {gen_loss:.4f} | Disc Loss: {disc_loss:.4f}")

    print(f"\nEpoch {epoch+1} Completed")
    print(f"Final Generator Loss: {gen_loss:.4f}")
    print(f"Final Discriminator Loss: {disc_loss:.4f}")

    generate_and_plot(generator, epoch+1, noise_seed)
```

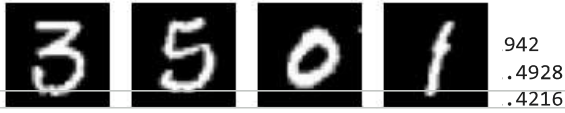
```
import matplotlib.pyplot as plt

def show_real_images(dataset, num_images=16):
    plt.figure(figsize=(4,4))

    for images in dataset.take(1):
        for i in range(num_images):
            plt.subplot(4,4,i+1)
            plt.imshow(images[i, :, :, 0] * 127.5 + 127.5, cmap='gray')
            plt.axis('off')
        plt.suptitle("Real MNIST Images")
        plt.show()

show_real_images(dataset)
```

Real MNIST Images



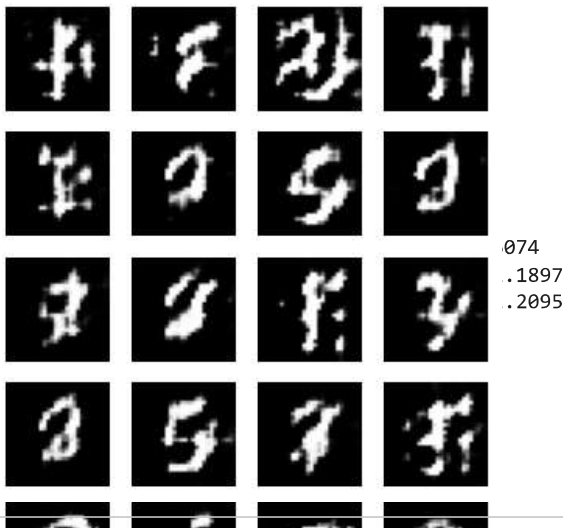
```
def show_generated_images(generator, num_images=16):
    noise = tf.random.normal([num_images, 100])
    generated_images = generator(noise, training=False)

    plt.figure(figsize=(4,4))
    for i in range(num_images):
        plt.subplot(4,4,i+1)
        plt.imshow(generated_images[i, :, :, 0] * 127.5 + 127.5, cmap='gray')
        plt.axis('off')

    plt.suptitle("Generated (Fake) Images")
    plt.show()

show_generated_images(generator)
```

Generated (Fake) Images



```
def compare_real_vs_fake(dataset, generator):
    noise = tf.random.normal([16, 100])
    generated_images = generator(noise, training=False)

    plt.figure(figsize=(8,4))

    # Real Images
    for images in dataset.take(1):
        for i in range(8):
            plt.subplot(2,8,i+1)
            plt.imshow(images[i, :, :, 0] * 127.5 + 127.5, cmap='gray')
            plt.axis('off')

    # Fake Images
    for i in range(8):
        plt.subplot(2,8,i+9)
        plt.imshow(generated_images[i, :, :, 0] * 127.5 + 127.5, cmap='gray')
        plt.axis('off')

    plt.suptitle("Top: Real Images | Bottom: Generated Images")
    plt.show()

compare_real_vs_fake(dataset, generator)
```

Final Discriminator Loss: 1.3181

Generated Images - Epoch 3

Top: Real Images | Bottom: Generated Images

